



Domain Driven Design - Java

Prof. Gilberto Alexandre das Neves
profgilberto.neves@fiap.com.br

Tratamento de Exceções

Uma exceção é um evento que interrompe o fluxo normal de execução de um programa. Em Java, as exceções são objetos que representam condições anormais, como erros de I/O, tentativas de acessar um índice fora dos limites de um *array*, ou operações aritméticas inválidas, como divisão por zero.

Dentro de **Exception**, há uma distinção importante:

- **Checked Exceptions:** São verificadas em tempo de compilação. O compilador exige que você trate essas exceções usando blocos **try-catch** ou declarando-as na assinatura do método com **throws**. Exemplos incluem *IOException*, *SQLException*.
- **Unchecked Exceptions (Runtime Exceptions):** Não são verificadas em tempo de compilação. Elas ocorrem em tempo de execução e não precisam ser explicitamente tratadas. Exemplos incluem *NullPointerException*, *ArrayIndexOutOfBoundsException*.

Blocos Try-Catch

Para tratar exceções em Java, utilizamos blocos **try-catch**. O código que pode lançar uma exceção é colocado dentro de um bloco **try**, e o bloco **catch** captura e trata a exceção.

```
try {  
    // Código que pode lançar uma exceção  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    // Tratamento da exceção  
    System.out.println("Erro: Divisão por zero!");  
}
```

Bloco Finally

O bloco **finally** é opcional e é usado para executar código que deve ser executado independentemente de uma exceção ser lançada ou não. É comum usar **finally** para liberar recursos, como fechar arquivos ou conexões de banco de dados.

```
try {  
    // Código que pode lançar uma exceção  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    // Tratamento da exceção  
    System.out.println("Erro: Divisão por zero!");  
} finally {  
    // Código que será executado sempre  
    System.out.println("Bloco finally executado.");  
}
```

O lançamento explícito de exceções em Java é feito usando a palavra-chave **throw**. Isso permite que você interrompa o fluxo normal do programa e "lance" uma exceção, indicando que ocorreu uma condição anômala ou erro que precisa ser tratado.

A sintaxe básica para lançar uma exceção explicitamente é:

```
throw new TipoDeExcecao("Mensagem de erro opcional");
```

Onde, **TipoDeExcecao** pode ser qualquer classe que estenda **Throwable**, normalmente uma subclasse de **Exception** ou **RuntimeException**.

Classe ExemploTryCatchFinally

```
public class ExemploTryCatchFinally {
    public static void main(String[] args) {
        try {
            String aux1 = JOptionPane.showInputDialog("Forneça o valor do 1º número."
                + " Faixa permitida de 1 à 50.");
            int num1 = Integer.parseInt(aux1);
            if (num1 < 1 || num1 > 50) {
                throw new Exception("Número informado fora da faixa permitida!");
            }
            String aux2 = JOptionPane.showInputDialog("Forneça o valor do 2º número");
            int num2 = Integer.parseInt(aux2);
            JOptionPane.showMessageDialog(null, "Soma = " + (num1 + num2));
            JOptionPane.showMessageDialog(null, "Subtração = " + (num1 - num2));
            JOptionPane.showMessageDialog(null, "Multiplicação = " + (num1 * num2));
            JOptionPane.showMessageDialog(null, "Divisão = " + (num1 / num2));
        } catch (ArithmeticException e) {
            JOptionPane.showMessageDialog(null, "Erro de divisão por zero!\n" + e.getMessage(),
                "Erro", JOptionPane.ERROR_MESSAGE);
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Erro de Conversão!\n" + e.getMessage(),
                "Erro", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Erro: " + e.getMessage(),
                "Erro", JOptionPane.ERROR_MESSAGE);
        } finally {
            JOptionPane.showMessageDialog(null, "Fim de Programa!");
        }
    }
}
```

O **throws** é utilizado na **assinatura de um método** para indicar que o método pode lançar uma ou mais exceções. Isso permite que as exceções sejam tratadas em um nível superior da pilha de chamadas, em vez de serem tratadas diretamente no método onde foram geradas.

Quando Usar throws?

- **Exceções Verificadas (Checked Exceptions):** Se um método pode lançar uma exceção que não é uma **RuntimeException** (ou seja, uma exceção verificada), você deve declarar isso usando **throws**. Se não declarar, o código não compilará.
- **Delegação de Tratamento:** Às vezes, faz mais sentido para um método delegar o tratamento de uma exceção para o código que o chama. Usar **throws** permite que você faça isso.

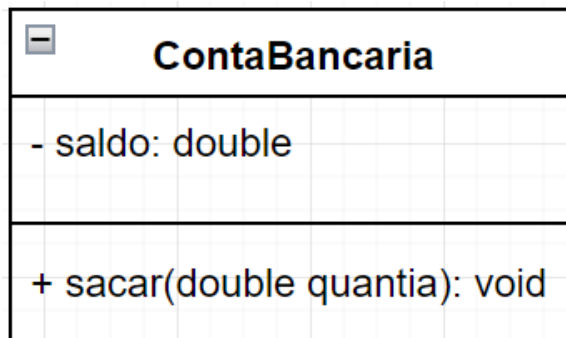
Exceções personalizadas são classes de exceção criadas por você, que estendem **Exception** ou **RuntimeException**. Elas permitem que você defina erros específicos ao contexto da sua aplicação, com mensagens e comportamentos apropriados.

Por Que Criar Exceções Personalizadas?

- **Clareza:** Exceções personalizadas podem tornar seu código mais legível e autodescritivo.
- **Contexto:** Fornece mais informações sobre o tipo de erro que ocorreu.
- **Modularidade:** Ajuda a organizar melhor o código de tratamento de exceções.

Classe ContaBancaria

Implemente a classe **ContaBancaria** como indicado pelo modelo UML abaixo.



Orientações:

- Definir apenas um construtor com parâmetro.
- Definir apenas o método *getter*.
- método **sacar()** deve subtrair o valor da quantia informada pelo saldo. Este método não deve permitir sacar uma quantia superior ao saldo atual (delegue o tratamento deste erro à classe **SaldoInsuficienteException**, que deve ser criada antes).

Em seguida, crie a classe de teste que deve criar um objeto da classe **ContaBancaria** (definindo um valor inicial para o saldo) e depois tentar realizar um saque de uma quantia superior ao saldo.

SaldoInsuficienteException e ContaBancaria

Implemente as classes no pacote **bean**:

```
public class SaldoInsuficienteException extends Exception{  
    public SaldoInsuficienteException(String message) {  
        super(message);  
    }  
}
```

```
public class ContaBancaria {  
    private double saldo;  
  
    public ContaBancaria(double saldo) {  
        this.saldo = saldo;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void sacar(double quantia) throws SaldoInsuficienteException {  
        if (quantia > saldo) {  
            throw new SaldoInsuficienteException("Saldo insuficiente "  
                + "para realizar o saque");  
        }  
        saldo -= quantia;  
    }  
}
```

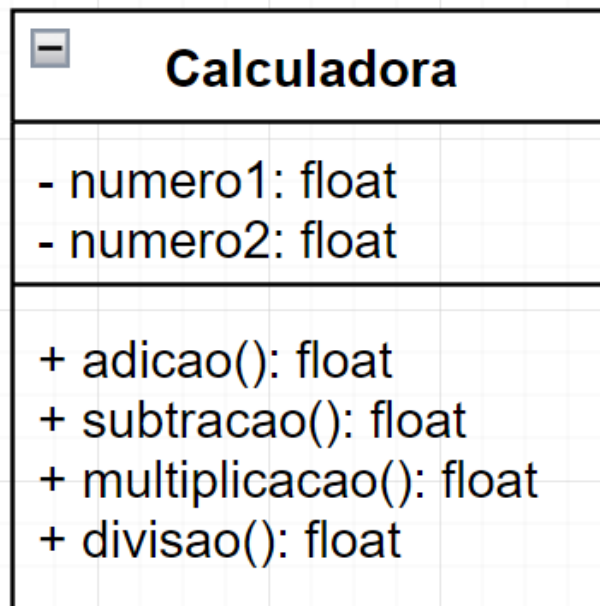
```
import br.com.fiap.bean.ContaBancaria;
import br.com.fiap.bean.SaldoInsuficienteException;

public class UsaContaBancaria {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria(1000.0);

        try {
            conta.sacar(1500.0);
        } catch (SaldoInsuficienteException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Praticando...

Implemente a classe **Calculadora** como indicado pelo modelo UML abaixo:



Orientações:

- Definir apenas um construtor vazio.
- Definir todos os *getters* e *setters*.
- método **divisao()** deve tratar o erro de divisão por zero (delegue o tratamento deste erro à classe **ArithmeticException**).

Implemente a classe **UsaCalculadora** com o método **main** para podermos criar objetos da classe criada anteriormente e testar seus atributos e métodos. Veja a seguir as orientações:

1. Peça para o usuário digitar dois números reais quaisquer (faça tratamento de erro para o que o usuário pode digitar).
2. Exiba um menu perguntando ao usuário que operação ele deseja realizar (soma, subtração, multiplicação ou divisão).
3. Utilize o método correspondente mediante a escolha do usuário e exiba o resultado da operação.
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

1. Qual das seguintes afirmações sobre o uso da palavra-chave **throw** em Java é correta?
 - a) O throw pode ser usado apenas para lançar exceções que estendem a classe RuntimeException
 - b) O throw é usado para capturar e tratar exceções em Java
 - c) O throw é utilizado para lançar explicitamente uma exceção em Java
 - d) O throw é usado para declarar que um método pode lançar uma exceção, mas não a lança diretamente

2. Qual das seguintes afirmações sobre a palavra-chave **throws** em Java é correta?
 - a) O throws é usado dentro de um bloco try para lançar exceções
 - b) O throws é usado para declarar que um método pode lançar uma ou mais exceções
 - c) O throws captura exceções lançadas por métodos e as trata automaticamente
 - d) O throws é uma palavra-chave opcional que pode ser usada em qualquer método, independentemente de ele lançar exceções ou não



Java como programar. Paul Deitel e Harvey Deitel. Pearson, 2011.

Java 8 – Ensino Didático : Desenvolvimento e Implementação de Aplicações. Sérgio Furgeri. Editora Érica, 2015.

Até breve!

1. Qual das seguintes afirmações sobre o uso da palavra-chave **throw** em Java é correta?
 - a) O throw pode ser usado apenas para lançar exceções que estendem a classe RuntimeException
 - b) O throw é usado para capturar e tratar exceções em Java
 - ☒ c) O throw é utilizado para lançar explicitamente uma exceção em Java
 - d) O throw é usado para declarar que um método pode lançar uma exceção, mas não a lança diretamente

2. Qual das seguintes afirmações sobre a palavra-chave **throws** em Java é correta?
 - a) O throws é usado dentro de um bloco try para lançar exceções
 - ☒ b) O throws é usado para declarar que um método pode lançar uma ou mais exceções
 - c) O throws captura exceções lançadas por métodos e as trata automaticamente
 - d) O throws é uma palavra-chave opcional que pode ser usada em qualquer método, independentemente de ele lançar exceções ou não